

Linux CLI vs. GUI Comparison Lab

Kali Linux Terminal vs. Ubuntu Desktop GUI

A companion hands-on lab for Cisco Networking Academy – Linux Essentials

Lab environment: VirtualBox 7.2.16 • Kali Linux VM (CLI) • Ubuntu Desktop 26.04 LTS VM (GUI)

How This Lab Works

You're learning Linux commands in the Linux Essentials course — this lab doesn't re-teach the commands or their flags. Instead, its whole purpose is to show you something that isn't obvious as a beginner: the command line and the graphical desktop are not two different systems. They are two different windows onto the exact same underlying Linux system. Every file, permission, process, and setting you touch with a command has a real, physical counterpart you can find and manipulate with a mouse — and vice versa.

For each topic below you will:

1. Run a command (or short set of commands) in the Kali Linux terminal and note the RESULT — what information it gave you or what changed.
2. Perform the equivalent action using Ubuntu Desktop's graphical tools and note what you see.
3. Read the Compare & Reflect note, then answer the reflection question(s) in your own words (out loud, in your head, or jot them down — whatever works for you).

Do the CLI step in Kali first, before looking at the GUI step — you'll get more out of the comparison if you predict what the GUI will show before you go look.

Environment Setup

Both machines already exist as VirtualBox VMs on this computer:

- **Kali Linux** — your CLI environment for this lab. Start it from VirtualBox Manager, log in, and open a Terminal (look for a black terminal icon in the taskbar/dock, or right-click the desktop and choose “Open Terminal Here”).
- **Ubuntu Desktop** — your GUI environment for this lab. Start it from VirtualBox Manager and log in with the account you created during installation.

Tip: if your monitor is large enough, resize both VM windows so you can see them side by side. That makes it much easier to compare a command's output against the GUI directly, instead of switching back and forth from memory.

A few commands below use `sudo` (administrator privileges). Use the password for the account you created when installing each VM. Nothing in this lab is destructive — you won't be asked to delete anything you need or modify system-critical files.

Topic 1: Where Am I? — Identity and Location

Before navigating anywhere, Linux always knows exactly who you are, where you are in the filesystem, and what machine you're on. This information is stored the same way whether you ask for it with text or a mouse.

Step 1 — CLI (Kali Linux terminal)

```
pwd
whoami
hostname
```

Run these one at a time in the Kali terminal. Note the three answers: your current directory path, your username, and the machine's hostname.

Step 2 — GUI (Ubuntu Desktop)

4. Open the Files app — look at the path/breadcrumb bar at the top; it shows where you currently are, the same concept as `pwd`.
5. Open Settings → About — this screen shows the device (host) name, matching `hostname`.
6. Click the top-right system menu (clock/quick settings area) — your account name is displayed there, matching `whoami`.

Compare & Reflect

`pwd`, `whoami`, and `hostname` aren't producing new information — they're reading the exact same identity and location data the desktop environment already knows and displays visually in different corners of the GUI. The CLI just lets you ask for one specific fact instantly instead of hunting for it across several settings screens.

Reflection question: Which method got you the hostname faster — typing `hostname`, or navigating to Settings → About? Why might that change once you're managing 20 servers instead of one?

Topic 2: Navigating the Filesystem

Every folder you click through in a file manager is a directory the shell can `cd` into. This section compares clicking versus typing to move around the same tree.

Step 1 — CLI (Kali Linux terminal)

```
cd ~
```

Go to your home directory.

```
ls
```

List what's here.

```
ls -la
```

List everything, including hidden files, with details.

```
cd Documents
```

Move into a subfolder (use any folder that exists).

```
cd ..
```

Move back up one level.

Watch how the prompt itself changes to reflect your current folder as you cd around.

Step 2 — GUI (Ubuntu Desktop)

7. Open the Files app — it opens to your Home folder by default, same as `cd ~`.
8. Double-click into Documents, then click the Back arrow (or click “Home” in the sidebar) to go back — same effect as `cd Documents` then `cd ..`
9. Press `Ctrl+H` (or View menu → Show Hidden Files) to reveal hidden files — compare this list to what `ls -la` showed you.

Compare & Reflect

Hidden files (anything starting with a dot, like `.bashrc`) aren't a CLI-only concept — the GUI hides them by default too, and `Ctrl+H` is doing the exact same job as the `-a` flag: toggling visibility of the same files, not revealing something the terminal could see and the GUI couldn't.

Reflection question: Did `ls -la` show any files in your home folder that you didn't expect? What do you think they're for?

Topic 3: Creating, Copying, Moving, and Deleting Files

File management is the most common task in either interface. This section builds a small test file and folder, then manipulates them both ways.

Step 1 — CLI (Kali Linux terminal)

```
mkdir cli_test
```

Create a new folder.

```
touch cli_test/notes.txt
```

Create an empty file inside it.

```
cp cli_test/notes.txt cli_test/notes_copy.txt
```

Copy the file.

```
mv cli_test/notes_copy.txt cli_test/renamed.txt
```

Rename it (mv doubles as rename).

```
rm cli_test/renamed.txt
```

Delete the copy.

After each command, run `ls cli_test` to see the folder's contents change in real time.

Step 2 — GUI (Ubuntu Desktop)

10. In Files, navigate to Home and open the `cli_test` folder you just created with `mkdir` — you'll see `notes.txt` sitting there, created entirely from the command line but fully visible and usable in the GUI.
11. Right-click inside the folder → New Folder, to create a second folder the GUI way.
12. Right-click `notes.txt` → Copy, then right-click empty space → Paste — same result as `cp`.
13. Right-click the copy → Rename — same result as `mv`.
14. Right-click it → Move to Trash.

Compare & Reflect

Notice the one real difference: `rm` in the terminal deletes immediately and permanently by default, while the GUI's delete sends the file to Trash first, where it can still be recovered. Same filesystem, same folder — but the two interfaces default to different levels of safety for the exact same action.

Reflection question: Why do you think the GUI defaults to a recoverable Trash while the terminal's `rm` does not? Which would you rather have as the default when scripting something?

Topic 4: Viewing File Contents

Reading a file's contents is another task with very different “feel” in each interface, even though the bytes being read never change.

Step 1 — CLI (Kali Linux terminal)

```
cat cli_test/notes.txt
```

Dump the whole file to the screen (it's empty right now, so try a system file instead:)

```
cat /etc/os-release
```

View a small text file describing the OS.

```
less /etc/os-release
```

View the same file one page at a time — press q to quit.

less is built for files too large to comfortably dump with cat; for a file this small the output looks identical either way.

Step 2 — GUI (Ubuntu Desktop)

15. In Files, open the file manager's location bar (Ctrl+L) and type /etc to browse to that folder, then double-click os-release to open it in the Text Editor app.
16. Scroll through it the same way you paged through it with less.

Compare & Reflect

cat, less, and the Text Editor are all just different ways of rendering identical file content. cat is instant but dumps everything at once; less lets you page through without loading a huge file into memory; the GUI editor adds scrolling, search, and the ability to edit and save — more convenient, but ultimately reading the same bytes.

Reflection question: If a file were 2 GB of log data, which of these three methods would you reach for, and why?

Topic 5: File Permissions

Permissions are one of the clearest examples of the CLI and GUI literally displaying the same underlying data in two formats: a text string versus a set of checkboxes.

Step 1 — CLI (Kali Linux terminal)

```
ls -l cli_test/notes.txt
```

Look at the permission string at the far left, e.g. -rw-r--r--.

```
chmod 640 cli_test/notes.txt
```

Change the permissions.

```
ls -l cli_test/notes.txt
```

Check the string again — see what changed.

The permission string breaks into: file type, owner (rwx), group (rwx), others (rwx).

Step 2 — GUI (Ubuntu Desktop)

17. In Files, right-click notes.txt → Properties → Permissions tab.

18. Look at the dropdowns/checkboxes for Owner, Group, and Others access — compare them to the string `chmod` just produced.
19. Change one of the dropdowns (e.g. set “Others” access to nothing) and click Close, then re-run `ls -l` in the terminal to see the string update to match your GUI change.

Compare & Reflect

This is the clearest 1-to-1 mapping in the whole lab: every checkbox/dropdown in the Properties → Permissions tab corresponds directly to one character in the `rwX` string. Changing it in the GUI rewrites the same permission bits `chmod` writes — there is no difference in the result, only in how you get there.

Reflection question: Which felt more intuitive for understanding what a permission actually means — reading `rw-r--r--`, or seeing labeled dropdowns? Which would you trust more when double-checking a sensitive file under time pressure?

Topic 6: Searching for Files and Data

Both interfaces can search — but not for the same things by default, which is a useful gap to notice as a beginner.

Step 1 — CLI (Kali Linux terminal)

```
find ~ -iname "*.txt"
```

Find every .txt file under your home folder by name.

```
grep -r "NAME" /etc/os-release
```

Search inside a file's contents for a word.

find searches filenames/paths; grep searches the text inside files.

Step 2 — GUI (Ubuntu Desktop)

20. In Files, press `Ctrl+F` (or click the search icon) and type `.txt` — compare the results to what `find` gave you.
21. Try searching for a word you know is only inside a file's content, not in any filename — notice that the Files app will not find it.

Compare & Reflect

This is a place where the CLI is genuinely more capable, not just differently presented: Ubuntu's default file search only matches filenames, while `grep` can search the actual text inside every file, recursively, in one command. The GUI simply doesn't expose that capability by default.

Reflection question: Can you think of a real situation where you'd need to find which file contains a certain word, not just a file with a certain name?

Topic 7: Users and Groups

Every account you can log into is a line of data on the system — the Settings app is just a friendlier view onto that same data.

Step 1 — CLI (Kali Linux terminal)

```
id
```

Show your user ID, group ID, and group memberships.

```
groups
```

List just the group names you belong to.

```
grep "$(whoami)" /etc/passwd
```

Find your account's line in the master user list.

/etc/passwd (despite the name) doesn't store your password — it stores account metadata: username, user ID, home folder, and default shell.

Step 2 — GUI (Ubuntu Desktop)

22. Open Settings → Users — you'll see your account listed with its type (Administrator or Standard).
23. Click your account to see its details — compare the account name shown here to the line `grep` pulled from `/etc/passwd`.

Compare & Reflect

The Settings → Users panel is a restricted, simplified view of the same account database `/etc/passwd` describes in full technical detail. Notice that, just like the terminal needs `sudo` to modify another user's account, the Settings app will ask for your password before letting you change anything — same underlying protection, enforced at both layers.

Reflection question: Why do you think Ubuntu shows “Administrator vs Standard” in the GUI instead of raw permission numbers, while the CLI shows you the raw ID numbers directly?

Topic 8: Processes and System Monitoring

A running program is a process whether you started it by typing a command or double-clicking an icon — both interfaces are looking at the same live process table.

Step 1 — CLI (Kali Linux terminal)

```
ps aux | less
```

List every running process with details. Press q to quit less.

```
top
```

Watch a live, auto-updating view of processes by CPU/memory use. Press q to quit.

Note a process name you recognize (e.g. bash, Xorg, gnome-shell) and its PID (process ID) number.

Step 2 — GUI (Ubuntu Desktop)

24. Open the System Monitor app (search for it in the Activities overview) → Processes tab.
25. Find the same process name you noted from ps/top, and compare its PID — it should be the identical number.
26. Click the %CPU column header to sort by CPU usage — compare this to what top showed you automatically.

Compare & Reflect

ps, top, and System Monitor are three different lenses on the exact same live process table the Linux kernel maintains. The PID numbers matching between the terminal and the GUI proves they aren't separate systems keeping separate records — there's only one process table, and both tools are just reading it.

Reflection question: System Monitor lets you end a process by clicking it and pressing a button, similar to a CLI kill command you'll learn about. Why might it be dangerous to end the wrong process either way?

Topic 9: Disk and Storage Usage

Free space, used space, and folder sizes are calculated from the same filesystem data no matter which tool asks for them.

Step 1 — CLI (Kali Linux terminal)

```
df -h
```

Show space used/free on each mounted filesystem, in human-readable form.

```
du -sh ~/Documents
```

Show the total size of one folder (substitute any folder that exists).

df looks at whole partitions; du adds up the size of files inside a folder.

Step 2 — GUI (Ubuntu Desktop)

27. Open the Disks app (search Activities for “Disks”) — select your virtual disk and compare the used/free space shown to what `df -h` reported.
28. In Files, right-click the Documents folder → Properties — compare the “Contents” size shown there to what `du -sh` reported.

Compare & Reflect

`df -h` and the Disks app both describe space at the whole-partition level; `du -sh` and a folder's Properties dialog both describe space at the individual-folder level. They're two different questions (“how full is the drive” vs “how big is this folder”) that happen to have both a CLI and a GUI way to ask them.

Reflection question: Did the Properties dialog take longer to calculate the folder size than `du -sh` did? Why might that be, given they're doing the same underlying work?

Topic 10: Package Management

Installing software is one of the biggest visible differences between a security-focused distro like Kali (CLI-first, `apt`) and a desktop distro like Ubuntu (GUI-first, Software Center) — but they're built on the same underlying package system.

Step 1 — CLI (Kali Linux terminal)

```
apt list --installed | less
```

See every package currently installed. Press `q` to quit.

```
apt search htop
```

Search for a package by name/description without installing it.

These are read-only/search commands — nothing gets installed or removed in this step.

Step 2 — GUI (Ubuntu Desktop)

29. Open the “Software” app (Ubuntu Software / App Center) on Ubuntu Desktop.
30. Use its search bar to look up the same program (e.g. `htop`) and open its listing page.
31. Compare the description shown in the app to the description `apt search` printed in the terminal.

Compare & Reflect

Ubuntu's Software app is a graphical front end built on top of the same APT package system apt uses directly — the same package names, versions, and descriptions, just presented with icons, screenshots, and a one-click Install button instead of a command you type out.

Reflection question: Kali Linux, being aimed at security professionals, doesn't ship a graphical software store by default the way Ubuntu does. Why do you think a CLI-first approach to installing tools might be preferred in that context?

Topic 11: Basic Network Information

Your VM's network identity — its IP address and connectivity — is visible both as raw text and as a graphical settings panel.

Step 1 — CLI (Kali Linux terminal)

```
ip a
```

List network interfaces and their IP addresses.

```
hostname -I
```

Print just this machine's IP address(es).

```
ping -c 4 google.com
```

Send 4 test packets out to check connectivity.

Look for an interface like enp0s3 with an inet address in the 10.0.2.x range — that's VirtualBox's default NAT network.

Step 2 — GUI (Ubuntu Desktop)

32. Open Settings → Network (or Wi-Fi, depending on adapter type) on Ubuntu Desktop.
33. Click the gear/settings icon next to the active connection — the IPv4 tab shows the address, matching what ip a reported.

Compare & Reflect

The IP address is identical in both places — same network configuration, read two ways. But notice ping has no default GUI equivalent in stock Ubuntu: testing connectivity by sending packets and timing responses is a task that stays CLI-only for most desktop users, which is common for diagnostic tools in general.

Reflection question: Why might a graphical OS choose to expose your IP address in Settings, but not build in a graphical ping tool?

Wrap-Up

Across all eleven topics, the same pattern held: the command line and the graphical desktop are not two different Linux systems living side by side — they are two interfaces reading and writing the exact same files, permissions, processes, users, and settings. Sometimes the GUI is simply friendlier (permissions, users, package browsing). Sometimes the CLI is genuinely more capable, not just faster (searching file contents with `grep`, scripting repeatable tasks, working over a remote connection with no desktop available at all — which is exactly why a tool-heavy, CLI-first distribution like Kali Linux exists in the first place).

Final Reflection

- Which single topic in this lab most changed how you think about what a GUI “actually is” under the hood?
- Now that you've seen both, which tasks do you think you'll personally prefer to do by command line going forward, and which by GUI?
- Metasploitable2 and Wazuh Manager, the other VMs on this system, don't have a graphical desktop installed at all — based on this lab, what would you expect to still be possible on them, and what would you lose?

End of lab.